C++ 并发三剑客future, promise和async

Posted on 2023-09-17 18:08:58 by 恋恋风辰 | In C++ | 原文链接 (/articlepage?id=2VWIJgH3zKEww0BpLnYQX0NMpQ9)

简介

本文介绍C++ 并发三剑客， future, promise以及async用法。这三个类是实现并发技术的关键，接下来详细介绍一下

async用法

`std::async` 是一个用于异步执行函数的模板函数，它返回一个 `std::future` 对象，该对象用于获取函数的返回值。

以下是一个使用 `std::async` 的示例：

```cpp
#include <iostream>
#include <future>
#include <chrono>

// 定义一个异步任务
std::string fetchDataFromDB(std::string query) {
    // 模拟一个异步任务，比如从数据库中获取数据
    std::this_thread::sleep_for(std::chrono::seconds(5));
    return "Data: " + query;
}

int main() {
    // 使用 std::async 异步调用 fetchDataFromDB
    std::future<std::string> resultFromDB = std::async(std::launch::async, fetchDataFromDB, "Data");

    // 在主线程中做其他事情
    std::cout << "Doing something else..." << std::endl;

    // 从 future 对象中获取数据
    std::string dbData = resultFromDB.get();
    std::cout << dbData << std::endl;

    return 0;
}
```

在这个示例中， `std::async` 创建了一个新的线程（或从内部线程池中挑选一个线程）并自动与一个 `std::promise` 对象相关联。 `std::promise` 对象被传递给 `fetchDataFromDB` 函数，函数的返回值被存储在 `std::future` 对象中。在主线程中，我们可以使用 `std::future::get` 方法从 `std::future` 对象中获取数据。注意，在使用 `std::async` 的情况下，我们必须使用 `std::launch::async` 标志来明确表明我们希望函数异步执行。

上面的例子输出

```
Doing something else...
Data: Data
```

async的启动策略

`std::async` 函数可以接受几个不同的启动策略，这些策略在 `std::launch` 枚举中定义。除了 `std::launch::async` 之外，还有以下启动策略：

1. `std::launch::deferred`：这种策略意味着任务将在调用 `std::future::get()` 或 `std::future::wait()` 函数时延迟执行。换句话说，任务将在需要结果时同步执行。
2. `std::launch::async | std::launch::deferred`：这种策略是上面两个策略的组合。任务可以在一个单独的线程上异步执行，也可以延迟执行，具体取决于实现。

默认情况下，`std::async` 使用 `std::launch::async | std::launch::deferred` 策略。这意味着任务可能异步执行，也可能延迟执行，具体取决于实现。需要注意的是，不同的编译器和操作系统可能会有不同的默认行为。

future的wait和get

`std::future::get()` 和 `std::future::wait()` 是 C++ 中用于处理异步任务的两个方法，它们的功能和用法有一些重要的区别。

1. **std::future::get()**:

`std::future::get()` 是一个阻塞调用，用于获取 `std::future` 对象表示的值或异常。如果异步任务还没有完成，`get()` 会阻塞当前线程，直到任务完成。如果任务已经完成，`get()` 会立即返回任务的结果。重要的是，`get()` 只能调用一次，因为它会移动或消耗掉 `std::future` 对象的状态。一旦 `get()` 被调用，`std::future` 对象就不能再被用来获取结果。

1. **std::future::wait()**:

`std::future::wait()` 也是一个阻塞调用，但它与 `get()` 的主要区别在于 `wait()` 不会返回任务的结果。它只是等待异步任务完成。如果任务已经完成，`wait()` 会立即返回。如果任务还没有完成，`wait()` 会阻塞当前线程，直到任务完成。与 `get()` 不同，`wait()` 可以被多次调用，它不会消耗掉 `std::future` 对象的状态。

总结一下，这两个方法的主要区别在于：

- `std::future::get()` 用于获取并返回任务的结果，而 `std::future::wait()` 只是等待任务完成。
- `get()` 只能调用一次，而 `wait()` 可以被多次调用。
- 如果任务还没有完成，`get()` 和 `wait()` 都会阻塞当前线程，但 `get()` 会一直阻塞直到任务完成并返回结果，而 `wait()` 只是在等待任务完成。

你可以使用std::future的wait_for()或wait_until()方法来检查异步操作是否已完成。这些方法返回一个表示操作状态的std::future_status值。

```
1. if(fut.wait_for(std::chrono::seconds(0)) == std::future_status::ready) {
2.     // 操作已完成
3. } else {
4.     // 操作尚未完成
5. }
```

将任务和future关联

`std::packaged_task` 和 `std::future` 是C++11中引入的两个类，它们用于处理异步任务的结果。

`std::packaged_task` 是一个可调用目标，它包装了一个任务，该任务可以在另一个线程上运行。它可以捕获任务的返回值或异常，并将其存储在 `std::future` 对象中，以便以后使用。

`std::future` 代表一个异步操作的结果。它可以用于从异步任务中获取返回值或异常。

以下是使用 `std::packaged_task` 和 `std::future` 的基本步骤：

1. 创建一个 `std::packaged_task` 对象，该对象包装了要执行的任务。
2. 调用 `std::packaged_task` 对象的 `get_future()` 方法，该方法返回一个与任务关联的 `std::future` 对象。
3. 在另一个线程上调用 `std::packaged_task` 对象的 `operator()` ，以执行任务。
4. 在需要任务结果的地方，调用与任务关联的 `std::future` 对象的 `get()` 方法，以获取任务的返回值或异常。

以下是一个简单的示例代码：

```
int my_task() {
    std::this_thread::sleep_for(std::chrono::seconds(5));
    std::cout << "my task run 5 s" << std::endl;
    return 42;
}

void use_package() {
    // 创建一个包装了任务的 std::packaged_task 对象
    std::packaged_task<int()> task(my_task);

    // 获取与任务关联的 std::future 对象
    std::future<int> result = task.get_future();

    // 在另一个线程上执行任务
    std::thread t(std::move(task));
    t.detach(); // 将线程与主线程分离，以便主线程可以等待任务完成

    // 等待任务完成并获取结果
    int value = result.get();
    std::cout << "The result is: " << value << std::endl;

}
```

在上面的示例中，我们创建了一个包装了任务的 `std::packaged_task` 对象，并获取了与任务关联的 `std::future` 对象。然后，我们在另一个线程上执行任务，并等待任务完成并获取结果。最后，我们输出结果。

我们可以使用 std::function 和 std::package_task 来包装带参数的函数。std::package_task 是一个模板类，它包装了一个可调用对象，并允许我们将其作为异步任务传递。

promise 用法

C++11引入了 `std::promise` 和 `std::future` 两个类，用于实现异步编程。 `std::promise` 用于在某一线程中设置某个值或异常，而 `std::future` 则用于在另一线程中获取这个值或异常。

下面是 `std::promise` 的基本用法：

```
#include <iostream>
#include <thread>
#include <future>

void set_value(std::promise<int> prom) {
    // 设置 promise 的值
    prom.set_value(10);
}

int main() {
    // 创建一个 promise 对象
    std::promise<int> prom;
    // 获取与 promise 相关联的 future 对象
    std::future<int> fut = prom.get_future();
    // 在新线程中设置 promise 的值
    std::thread t(set_value, std::move(prom));
    // 在主线程中获取 future 的值
    std::cout << "Waiting for the thread to set the value...\n";
    std::cout << "Value set by the thread: " << fut.get() << '\n';
    t.join();
    return 0;
}
```

程序输出

```
Waiting for the thread to set the value...
promise set value successValue set by the thread:
10
```

在上面的代码中，我们首先创建了一个 `std::promise<int>` 对象，然后通过调用 `get_future()` 方法获取与之相关联的 `std::future<int>` 对象。然后，我们在新线程中通过调用 `set_value()` 方法设置 `promise` 的值，并在主线程中通过调用 `fut.get()` 方法获取这个值。注意，在调用 `fut.get()` 方法时，如果 `promise` 的值还没有被设置，则该方法会阻塞当前线程，直到值被设置为止。

除了 `set_value()` 方法外， `std::promise` 还有一个 `set_exception()` 方法，用于设置异常。该方法接受一个 `std::exception_ptr` 参数，该参数可以通过调用 `std::current_exception()` 方法获取。下面是一个例子：

```cpp
#include <iostream>
#include <thread>
#include <future>

void set_exception(std::promise<void> prom) {
    try {
        // 抛出一个异常
        throw std::runtime_error("An error occurred!");
    } catch(...) {
        // 设置 promise 的异常
        prom.set_exception(std::current_exception());
    }
}

int main() {
    // 创建一个 promise 对象
    std::promise<void> prom;
    // 获取与 promise 相关联的 future 对象
    std::future<void> fut = prom.get_future();
    // 在新线程中设置 promise 的异常
    std::thread t(set_exception, std::move(prom));
    // 在主线程中获取 future 的异常
    try {
        std::cout << "Waiting for the thread to set the exception...\n";
        fut.get();
    } catch(const std::exception& e) {
        std::cout << "Exception set by the thread: " << e.what() << '\n';
    }
    t.join();
    return 0;
}
```

上述代码输出

```
Waiting for the thread to set the exception...
Exception set by the thread: An error occurred!
```

当然我们使用promise时要注意一点，如果promise被释放了，而其他的线程还未使用与promise关联的future，当其使用这个future时会报错。如下是一段错误展示

```
void use_promise_destruct() {
    std::thread t;
    std::future<int> fut;
    {
        // 创建一个 promise 对象
        std::promise<int> prom;
        // 获取与 promise 相关联的 future 对象
        fut = prom.get_future();
        // 在新线程中设置 promise 的值
         t = std::thread(set_value, std::move(prom));
    }
    // 在主线程中获取 future 的值
    std::cout << "Waiting for the thread to set the value...\n";
    std::cout << "Value set by the thread: " << fut.get() << '\n';
    t.join();
}
```

随着局部作用域 `}` 的结束， `prom` 可能被释放也可能会被延迟释放，
如果立即释放则 `fut.get()` 获取的值会报 `error_value` 的错误。

共享类型的future

当我们需要多个线程等待同一个执行结果时，需要使用std::shared_future

以下是一个适合使用 `std::shared_future` 的场景，多个线程等待同一个异步操作的结果：

假设你有一个异步任务，需要多个线程等待其完成，然后这些线程需要访问任务的结果。在这种情况下，你可以使用 `std::shared_future` 来共享异步任务的结果。

```cpp
void myFunction(std::promise<int>&& promise) {
    // 模拟一些工作
    std::this_thread::sleep_for(std::chrono::seconds(1));
    promise.set_value(42); // 设置 promise 的值
}

void threadFunction(std::shared_future<int> future) {
    try {
        int result = future.get();
        std::cout << "Result: " << result << std::endl;
    }
    catch (const std::future_error& e) {
        std::cout << "Future error: " << e.what() << std::endl;
    }
}

void use_shared_future() {
    std::promise<int> promise;
    std::shared_future<int> future = promise.get_future();

    std::thread myThread1(myFunction, std::move(promise)); // 将 promise 移动到线程中

    // 使用 share() 方法获取新的 shared_future 对象

    std::thread myThread2(threadFunction, future);

    std::thread myThread3(threadFunction, future);

    myThread1.join();
    myThread2.join();
    myThread3.join();
}
```

在这个示例中，我们创建了一个 `std::promise<int>` 对象 `promise` 和一个与之关联的 `std::shared_future<int>` 对象 `future` 。然后，我们将 `promise` 对象移动到另一个线程 `myThread1` 中，该线程将执行 `myFunction` 函数，并在完成后设置 `promise` 的值。我们还创建了两个线程 `myThread2` 和 `myThread3` ，它们将等待 `future` 对象的结果。如果 `myThread1` 成功地设置了 `promise` 的值，那么 `future.get()` 将返回该值。这些线程可以同时访问和等待 `future` 对象的结果，而不会相互干扰。

但是大家要注意，如果一个 `future` 被移动给两个 `shared_future` 是错误的。

```
1. void use_shared_future() {
2.     std::promise<int> promise;
3.     std::shared_future<int> future = promise.get_future();
4.
5.     std::thread myThread1(myFunction, std::move(promise)); // 将 promise 移动到线程中
6.
7.     std::thread myThread2(threadFunction, std::move(future));
8.     std::thread myThread3(threadFunction, std::move(future));
9.
10.    myThread1.join();
11.    myThread2.join();
12.    myThread3.join();
13. }
```

这种用法是错误的，一个 `future` 通过隐式构造传递给 `shared_future` 之后，这个 `shared_future` 被移动传递给两个线程是不合理的，因为第一次移动后 `shared_future` 的生命周期被转移了，接下来 `myThread3` 构造时用的 `std::move(future)` future已经失效了，会报错，一般都是 `no state` 之类的错误。

异常处理

`std::future` 是C++的一个模板类，它用于表示一个可能还没有准备好的异步操作的结果。你可以通过调用 `std::future::get` 方法来获取这个结果。如果在获取结果时发生了异常，那么 `std::future::get` 会重新抛出这个异常。

以下是一个例子，演示了如何在 `std::future` 中获取异常：

```
#include <iostream>
#include <future>
#include <stdexcept>
#include <thread>

void may_throw()
{
    // 这里我们抛出一个异常。在实际的程序中，这可能在任何地方发生。
    throw std::runtime_error("Oops, something went wrong!");
}

int main()
{
    // 创建一个异步任务
    std::future<void> result(std::async(std::launch::async, may_throw));

    try
    {
        // 获取结果（如果在获取结果时发生了异常，那么会重新抛出这个异常）
        result.get();
    }
    catch (const std::exception &e)
    {
        // 捕获并打印异常
        std::cerr << "Caught exception: " << e.what() << std::endl;
    }

    return 0;
}
```

在这个例子中，我们创建了一个异步任务 `may_throw` ，这个任务会抛出一个异常。然后，我们创建一个 `std::future` 对象 `result` 来表示这个任务的结果。在 `main` 函数中，我们调用 `result.get()` 来获取任务的结果。如果在获取结果时发生了异常，那么 `result.get()` 会重新抛出这个异常，然后我们在 `catch` 块中捕获并打印这个异常。

上面的例子输出

```
Caught exception: Oops, something went wrong!
```

线程池

我们可以利用上面提到的 `std::packaged_task` 和 `std::promise` 构建线程池，提高程序的并发能力。
先了解什么是线程池：

线程池是一种多线程处理形式，它处理过程中将任务添加到队列，然后在创建线程后自动启动这些任务。线程池线程都是后台线程。每个线程都使用默认的堆栈大小，以默认的优先级运行，并处于多线程单元中。如果某个线程在托管代码中空闲（如正在等待某个事件）,则线程池将插入另一个辅助线程来使所有处理器保持繁忙。如果所有线程池线程都始终保持繁忙，但队列中包含挂起的工作，则线程池将在一段时间后创建另一个辅助线程但线程的数目永远不会超过最大值。超过最大值的线程可以排队，但他们要等到其他线程完成后才启动。

线程池可以避免在处理短时间任务时创建与销毁线程的代价，它维护着多个线程，等待着监督管理者分配可并发执行的任务，从而提高了整体性能。

下面是我提供的一套线程池源码，目前用在公司的项目中

```
#ifndef __THREAD_POOL_H__
#define __THREAD_POOL_H__

#include <atomic>
#include <condition_variable>
#include <future>
#include <iostream>
#include <mutex>
#include <queue>
#include <thread>
#include <vector>

class ThreadPool  {
public:
    ThreadPool(const ThreadPool&) = delete;
    ThreadPool&        operator=(const ThreadPool&) = delete;

    static ThreadPool& instance() {
        static ThreadPool ins;
        return ins;
    }

    using Task = std::packaged_task<void()>;


    ~ThreadPool() {
        stop();
    }

    template <class F, class... Args>
    auto commit(F&& f, Args&&... args) -> std::future<decltype(f(args...))> {
        using RetType = decltype(f(args...));
        if (stop_.load())
            return std::future<RetType>{};

        auto task = std::make_shared<std::packaged_task<RetType()>>(
            std::bind(std::forward<F>(f), std::forward<Args>(args)...));

        std::future<RetType> ret = task->get_future();
        {
            std::lock_guard<std::mutex> cv_mt(cv_mt_);
            tasks_.emplace([task] { (*task)(); });
```

```
43.         }
44.         cv_lock_.notify_one();
45.         return ret;
46.     }
47.
48.     int idleThreadCount() {
49.         return thread_num_;
50.     }
51.
52. private:
53.     ThreadPool(unsigned int num = 5)
54.         : stop_(false) {
55.             {
56.                 if (num < 1)
57.                     thread_num_ = 1;
58.                 else
59.                     thread_num_ = num;
60.             }
61.             start();
62.     }
63.     void start() {
64.         for (int i = 0; i < thread_num_; ++i) {
65.             pool_.emplace_back([this]() {
66.                 while (!this->stop_.load()) {
67.                     Task task;
68.                     {
69.                         std::unique_lock<std::mutex> cv_mt(cv_mt_);
70.                         this->cv_lock_.wait(cv_mt, [this] {
71.                             return this->stop_.load() || !this->tasks_.empty();
72.                         });
73.                         if (this->tasks_.empty())
74.                             return;
75.
76.                         task = std::move(this->tasks_.front());
77.                         this->tasks_.pop();
78.                     }
79.                     this->thread_num_--;
80.                     task();
81.                     this->thread_num_++;
82.                 }
83.             });
84.         }
```

```
    }
    void stop() {
        stop_.store(true);
        cv_lock_.notify_all();
        for (auto& td : pool_) {
            if (td.joinable()) {
                std::cout << "join thread " << td.get_id() << std::endl;
                td.join();
            }
        }
    }

private:
    std::mutex              cv_mt_;
    std::condition_variable  cv_lock_;
    std::atomic_bool        stop_;
    std::atomic_int         thread_num_;
    std::queue<Task>        tasks_;
    std::vector<std::thread> pool_;
};

#endif  // !__THREAD_POOL_H__
```

总结

本文介绍了如何使用future, promise以及async用法

视频链接

https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290 (https://space.bilibili.com/271469206/channel/collectiondetail?sid=1623290)

源码链接

https://gitee.com/secondtonone1/boostasio-learn (https://gitee.com/secondtonone1/boostasio-learn)

浏览(3940) (JavaScript:void(0);) 评论(0) (JavaScript:void(0);) ♡喜欢(563) (JavaScript:void(0);)

热门评论

请输入正文

评论

没有更多评论了,点击收起 ≪ (JavaScript:void(0);)

热门文章

slice介绍和使用

♡喜欢(521) 浏览(1546)

C++ 类的继承封装和多态

♡喜欢(588) 浏览(2196)

windows环境搭建和vscode配置

♡喜欢(587) 浏览(1595)

解密定时器的实现细节

♡喜欢(566) 浏览(1449)

Linux环境搭建和编码

♡喜欢(594) 浏览(5076)

最新评论

C++ 类的继承封装和多态

gzy:谢谢老师，写的真的很好，本来对这些新的关键字都很模糊，几篇下来配合着ai记着笔记一下子就清晰了

C++ 右值引用与移动构造函数

李程浩:1

C++ 线程管控

secondtonone1:好的，已修复，写的时候马虎了，谢谢指正。

线程基础

secondtonone1:谢谢，笔误已修复

再谈单例模式

secondtonone1:是的，C++11以后返回局部static变量对象能保证线程安全了。

线程安全的无锁栈

secondtonone1:谢谢支持，如果pop的次数大于push的次数是会让线程处于重试的，这个是测试用例，必须满足push和pop的次数相同，实际情况不会这么使用。栈的设计没有问题。

创建项目和编译

secondtonone1:谢谢支持

visual studio配置boost库

secondtonone1:是的，您说的是对的，windows平台要采用msvc方式。

C++ 类的拷贝构造、赋值运算、单例模式

secondtonone1:好的，已修复。

asio多线程模式IOThreadPool

secondtonone1:这么优秀吗

asio实现http服务器

secondtonone1:我认为IO复用节省了线程资源，不用像传统服务器一样为每个连接单独创立线程，而是统一通过epoll或者iocp通知。

类和对象

陈宇航:支持！！！！

解决博客回复区被脚本注入的问题

secondtonone1:走到现在我忽然明白一个道理，无论工作也好生活也罢，最重要的是开心，即使一份安稳的工作不能给我带来事业上的积累也要合理的舍弃，所以我还是想去做喜欢的方向。

泛型算法的定制操作

secondtonone1:lambda和bind是C11新增的利器，善于利用这两个机制可以极大地提升编程安全性和效率。

双链表实现LRU算法

secondtonone1:双链表插入和删除节点是本篇的难点，多多练习即可。

构造函数

secondtonone1:构造函数是类的基础知识，要着重掌握

面试题汇总(一)

secondtonone1:看到网络上经常提问的go的问题，做了一下汇总，结合自己的经验给出的答案，如有纰漏，望指正批评。

使用mongo

secondtonone1:mongo操作是程序员必备的基础技能，包括增删改查，以及较为复杂的查询，分组查询，条件

查询，模糊查询，文档内查询等，本文结合自己实际开发经验做了整理。

interface应用

secondtonone1:interface是万能类型，但是使用时要转换为实际类型来使用。interface丰富了go的多态特性，也降低了传统面向对象语言的耦合性。

golang 函数介绍

secondtonone1:函数是go中的一等公民，作为新兴语言，go摒弃了面向对象的一些糟粕，采取接口方式编程，而接口方式编程都是基于函数的，参数为interface，进而达到泛型作用，比如sort排序，只需要传入的参数满…

堆排序

secondtonone1:堆排序非常实用，定时器就是这个原理制作的。

Linux环境搭建和编码

恋恋风辰:Linux环境下go的安装比较简单，可以不用设置GOPATH环境变量，后期我们学习go mod 之后就拜托了go文件目录的限制了。

slice介绍和使用

恋恋风辰:切片作为引用类型极大的提高了数据传递的效率和性能，但也要注意切片的浅拷贝隐患，算是一把双刃剑，这世间的常态就是在两极之间寻求一种稳定。

其他链接

GitHub (https://github.com)
个人博客 (https://www.limerence2017.com)
博客园 (https://www.cnblogs.com/secondtonone1/)
CSDN (https://blog.csdn.net/secondtonone1)

B站



个人微信



恋恋风辰

北京 朝阳



扫一扫上面的二维码图案，加我为朋友。

个人公众号



本站累计访问量204866次

本网站内容全部为原创，谢绝转载。

京公网安备 11010502047625号 (http://www.beian.gov.cn/portal/registerSystemInfo?recordcode=11010502047625)

京ICP备2021033081号 (https://beian.miit.gov.cn/)